

レンタルスタジオサーチアプリ 企画・設計アーカイブ

これはアーカイブ資料です。本アプリは、同じ開発者が先に運用していた「釣行AIアプリ」(fishing-ai-app)のアーキテクチャ・思想をほぼそのまま流用して、対象ドメインを「釣り場」から「ダンス・ヨガのレンタルスタジオ」に置き換える形で立ち上げられた。そのため通常の企画書・要件定義書のようなゼロからの起点は無く、「何を釣行AIアプリから引き継ぎ、何を変えたか」という変換の記録そのものが企画の実体になっている。本書はその変換方針と、その後のスコアリング刷新の経緯を記録として残す目的で保管しており、**現在の正確な仕様ではない箇所**（当初のスコア式・当初のデータ設計等）を含む。

現状の正しい情報は以下を参照:

- ・機能の全体像・使い方 → アプリまるわかりガイド.html
- ・Lambda関数単位の仕様 → 詳細設計.html
- ・ファイル構成・依存関係 → ファイル構成・依存関係.html
- ・AWSサービス構成 → AWS構成・サービス連携.html
- ・デプロイ手順 → README.md

1. 背景・課題

釣行AIアプリのAnthropic API・AWSアカウントを別アプリでも使い回せるかという相談から、「このアプリの基盤をほぼそのまま参考にしたレンタルスタジオサーチアプリを作りたい」という要望に発展した。ダンス・ヨガスタジオという対象を選んだのは、釣行AIアプリと同じ「実在する場所をGoogle Places APIで発見し、AIが説明・推薦理由を生成する」というパターンがそのまま当てはまるドメインだったため。

想定ユーザー

- ・ダンス・ヨガの個人練習者、初心者～中級者
- ・スタジオ選びに時間をかけたくない層

2. スコープ決定: 小さいMVPか、最初から full parity か

計画段階で「釣行AIアプリと同等の機能を最初から実装する」か「小さいMVPから始めて段階的に拡張する」かを検討した。釣行AIアプリ自体は段階的に機能を積み上げてきた経緯があることを説明した上で選択を仰いだところ、**「釣行AIアプリと同等の機能を最初から」**という方針が明確に選ばれた。理由は、釣行AIアプリという既に検証済みのアーキテクチャ・コードベースを流用する以上、ゼロから機能を段階検証する必要性が薄いため。この結果、認証（Cognito+Google）・AI相談・投稿編集・コスト保護レート制限・reason使い回しキャッシュといった「あると良い」機能も含め、v1から全て実装する方針で進めることになった。

3. ドメイン変換の対応表（初期案）

釣行AIアプリ	今回のアプリ
Spots（釣り場）	Studios（ダンス・ヨガスタジオ）
fishTypes（釣れる魚種）	facilityTags（設備タグ: 鏡張り・フローリング・音響設備等）
天気・潮汐スコア	口コミ評価（rating）・レビュー数（user_ratings_total）
discover_spots.py	discover_studios.py（同じくPlaces Text Search、クエリを"ダンススタジオ レンタル"等に変更）
釣果投稿（Posts）	レビュー投稿（Posts、写真+コメント+★評価）
AI相談（釣り・エサ）	AI相談（スタジオ選び・練習用途の相談）
釣具店検索（除外+別枠）	v1では対象外（明確な代替ドメインが無いため）

4. スコアロジックの変遷

当初案（釣行AIアプリの式をそのまま移植）

```
score = rating_score * 0.4      # Placesのrating(0-5)を0-100に正規化
      + popularity_score * 0.2  # user_ratings_totalを正規化
      + appeal_score * 0.2      # estimate_studio_appeal()のヒューリスティック
      - dist_norm * 0.1         # 基準地点（東京駅）からの距離
      - cost_norm * 0.1         # 常に0（Places APIが料金を返さないため）
```

この式は「距離」と「費用」が釣行AIアプリの構造をそのまま踏襲しており、特に距離は「基準地点（東京駅）からの直線距離」という、全国のスタジオを対象にする本アプリにとってはあまり意味の無い指標だった。費用も常に0固定で、実質機能していなかった。

刷新後（現行）

「釣りと違ってスタジオは天気は左右されず、一度付けたスコアがあまり変わらない」という指摘をきっかけに、スコア項目を見直した。

項目	変更内容	理由
距離	基準地点からの距離 → 最寄り駅からのアクセス（Places Nearby Search）	ユーザーの現在地に依存しない、スタジオ自体の固有の通いやすさを表す指標にするため
収容人数	新規追加（capacityCategory）	Claudeによる店名・住所からのヒューリスティック推測（設備タグ推測と同方式）。将来のエリア+人数選択機能の土台
料金	常に0 → 公式サイトを取得しClaudeが実際に書かれた金額を抽出	Places APIが料金を返さないため。サイトに無ければ「不明」とし、AIが金額を作り出すことはしない設計

既存スタジオ（スコアリング刷新前に発見済みだった分）は backend/scripts/backfill_studio_details.py で 後追いバックフィルした。最新の式・実装は詳細設計.htmlを参照。

5. AWSアーキテクチャの決定

「同じAWSアカウントを使うか、別アカウントか」を検討し、**釣行AIアプリと同じAWSアカウント（630956767121）に、完全に独立したCloudFormationスタック（studio-search-app）として構築する**方針を選択した。理由は、口座・請求を分ける必要性が薄く、同一アカウント内でもスタック単位で完全に隔離できるため。ただしCognito User Pool・SSMパラメータパス・Cloudflare Worker名は、両アプリの運用が互いに影響しないよう明確に分離した。

6. 実装中に発覚した不具合とその対応

Lambda関数名の衝突

初回デプロイ時、多くのLambda関数に釣行AIアプリと同じFunctionName

(getRecommendationsHandler等)を固定指定していたため、Lambda関数名の アカウント+リージョン一意制約に反して既存関数と衝突し、AWS::EarlyValidation::ResourceExistenceCheckでデプロイが失敗した。全LambdaのFunctionName指定を削除し、CloudFormationに自動採番させる形に修正して解消した。

フロントエンドの地図が表示されない

デプロイ後、地図ページが表示されない不具合が発生。当初はGoogle Cloud Console側の APIキー制限 (HTTPリファラー) が原因と推測したが、実際にブラウザが送信していた リクエストを調査したところ、GitHub SecretsのVITE_GOOGLE_MAPS_KEYに VITE_GOOGLE_MAPS_KEY=という変数名ごと値が登録されていたことが真因だった (.envファイルの行をそのままコピペしたことによる人為的ミス)。値をキー本体のみに 修正して解消した。

7. フロントエンド配信の追加: Cloudflare Workers

当初はAWS (CloudFront) 単独で構築したが、釣行AIアプリと同様に「独自ドメイン取得前の 見た目の良い無料URL」 としてCloudflare Workersへの並行デプロイを追加した。Worker名をstudio-search-app (釣行AIアプリはryu-chan-fish) と 明確に別名にし、GitHub Secretsもリポジトリごとに別登録することで、互いのWorkerを 誤って上書きするリスクを排除している。

8. 将来拡張アイデア (未着手)

- エリア・人数を選択してからAI分析を実行するモード (「現在地から探す」は即時系として現状のまま維持)。 収容人数目安 (capacityCategory) は既に実装済みで、この機能の土台として先に追加した
- AI分析または現在地から見つけたスタジオを実際に予約できる機能
- アプリ内レビューの★評価を、独自の信頼シグナルとしてスコアへ組み込む (Google側の評価とは別軸)
- 営業時間の柔軟さ (Places APIのopening_hours) をスコアに反映

9. まとめ

本プロダクトは「釣行AIアプリのアーキテクチャを、別ドメインに移植する」という形で 立ち上がった。企画段階から「実在データ (Google Places API) とAIによる説明生成

を分離し、AIには存在しないスタジオ・存在しない料金を作らせない」という設計思想を引き継いでおり、スコアリング刷新（駅距離・収容人数・料金抽出）でもこの原則は一貫して守られている。